

Technical Defense Report 3



Queen Amann

by Quatre-Quarts

Clémence GOUTAUDIER
Mary Bruny NGUEDIA NGUIAZON
Yasmine DGHOUGH
Matteo PARNIERE--BOSQUEZ
Céleste LECAMP

May 2026

1 Introduction

- 1.1 Context of the project
- 1.2 Objectives of the final technical defence

2 Project Overview and Final State

- 2.1 Summary of what was achieved
- 2.2 Technologies and tools used

3 Team Organisation and Task Distribution

- 3.1 Team presentation
- 3.2 Final task distribution and responsibilities

4 Level Design

- 4.1 A1 Level "Earth"
- 4.2 A2 Level "Air"
- 4.3 B1 Level "Fire"
- 4.4 B2 Level "Water"
- 4.5 The Lobby

5 Technical Implementations

- 5.1 Multiplayer and networking
- 5.2 Room-based session system
- 5.3 Player mechanics
- 5.4 Cooperative mechanics
- 5.5 Environmental mechanics
- 5.6 World navigation
- 5.7 Narrative systems
- 5.8 Artificial intelligence
- 5.9 User interface and menus

6 Visuals and Audio

- 6.1 Visual direction and final assets
- 6.2 Sound effects and original music

7 Website Development

8 Project Packaging and Deliverables

9 Integration and Collaboration Challenges

- 9.1 GitHub workflow and lessons learned
- 9.2 Merging independent codebases

10 Objectives Achieved vs Initial Roadmap

11 What We Learnt

12 Conclusion

1. Introduction

1.1 Context of the project

This project is part of a collaborative academic assignment focused on the design and development of a video game. The objective was to apply theoretical and technical knowledge acquired during the curriculum to a concrete, multidisciplinary project involving programming, game design, artificial intelligence, networking, visual design, and project management.

The project was developed as a team of five, combining research, experimentation, and iterative implementation across three development phases. Attention was given throughout to collaboration, technical feasibility, and creative originality. In addition to the game itself, the project also includes the development of a dedicated website, full technical documentation, an operating file and a user's manual, and a packaged deliverable, ensuring a complete and professional presentation of both the creative and technical aspects of the work.

1.2 Objectives of the final technical defence

This third and final technical defence marks the conclusion of the Queen Amann project. Unlike the two previous defences, which focused on validating technical foundations and demonstrating incremental progress, this defence presents a fully playable and complete game.

Its main objectives are to present the final state of the project in its entirety, detailing every system that has been implemented across the three development phases, to reflect honestly on the challenges encountered during the final integration phase, particularly regarding collaborative coding and version control, to demonstrate the cooperative gameplay experience that sits at the heart of Queen Amann, including all four worlds, their unique mechanics, and the overarching narrative, and finally to present all accompanying deliverables, including the website, the dossier d'exploitation, the installation procedure, and the packaged USB version of the game.

This defence is not only a technical presentation but a reflection on everything the team has learnt, individually and collectively, throughout this project.

2. Project Overview and Final State

2.1 Summary of what was achieved

Queen Amann is a 2D cooperative puzzle-platformer developed in Python using the PyGame library. The game is now complete and fully playable. Two players connect online to a shared

server, choose their character, and embark together on a quest to save Queen Amann by recovering the ingredients of a legendary recipe capable of breaking a mysterious curse.

The game features four distinct worlds {Earth, Air, Fire, and Water} each with its own visual identity, original music, environmental mechanics, and level design. Together these worlds form a coherent and progressive gameplay experience, from the underground maze of the Earth realm to the wind-swept floating ruins of the Air world, the lava-filled Church of Fire, and the cascading waterfall caves of the Water realm. Players navigate between worlds through a central hub called the Lobby, which also serves as the narrative entry point of the game.

At the heart of the experience is cooperation. The game cannot be completed alone. Two players must communicate, coordinate their actions, and use the unique abilities of their respective characters to progress. This cooperative philosophy is embedded in every system of the game, from the pressure plate and gate mechanics that require both players to act simultaneously, to the fire immunity of the character Tonatiuh which forces players to divide roles in the Fire world. Every world has been designed so that the two characters genuinely need each other.

The game's narrative concludes at the Queen statue in the Lobby, where players must enter the correct ingredient quantities discovered at the end of each level. If the recipe is complete and correct, the Queen is saved. This final interaction ties together every level, every chest, and every sign the players encountered throughout their journey, giving the game a clear and satisfying conclusion.

Since the second technical defence, the following major elements have been completed and integrated into a single, stable codebase:

- All four worlds fully designed, mapped, and playable
- A fully functional online multiplayer system with room creation and joining
- Level-specific environmental mechanics (air currents, moving platforms, lava immunity, hazard zones)
- A cooperative gate and bridge system requiring player coordination
- A complete settings and help menu accessible during gameplay
- Full audio integration with original music composed for each world and sound effects for all player actions
- A world navigation system with animated transitions between maps
- A narrative finale with the Queen statue and interactive recipe popup
- A fully updated and responsive website with gameplay videos and all project documents
- A packaged USB version of the game ready for delivery

2.2 Technologies and tools used

The game was developed using Python as the main programming language, with PyGame handling rendering, input, physics, and game logic. The multiplayer server is implemented in Python using the Sockets and Threading libraries, with JSON used for all data serialisation between clients and server. The server is deployed online via the Koyeb hosting platform, meaning the game requires an internet connection to be played.

Visual Studio Code and PyCharm were used as development environments throughout the project. GitHub served as the version control and collaboration platform, and its correct use became one of the most important lessons of the final development phase. Notion was used for project organisation and task tracking, whilst WhatsApp and Discord supported day-to-day team communication.

Level design was handled through a custom level editor developed by Matteo, which exports maps in JSON format and supports both collidable and decorative elements on a 32×32 grid. All visual assets were created by hand using Procreate, and all music was composed and recorded originally by Matteo. Sound effects were sourced from online libraries, whilst no generative artificial intelligence, stock image libraries, or pre-made audio banks were used for any visual or musical element of the project.

3. Team Organisation and Task Distribution

3.1 Team presentation

Our company is called 4.¼ (Quatre-Quarts). This name reflects both our team spirit and our way of working: like a recipe, each member brings an essential part to create a balanced and coherent project. Quatre-Quarts was founded in September 2025 as part of our academic programme. In October of the same year, we welcomed a fifth member, completing the team that has worked together through every stage of this project.

The core values of Quatre-Quarts are creativity, both artistic and technical, and curiosity. We love understanding how things work, experimenting on our own, and learning by doing. This project was our first video game, both individually and collectively, and we chose to see that as a strength rather than a limitation. Everything was new, everything had to be built from scratch, and that challenge ultimately became the most rewarding part of the experience.

The first defence report described the team as motivated by creativity and curiosity, and noted that none of its members had prior experience in video game development, either individually or collectively. Rather than treating this as a weakness, the team chose to frame it as a strength: because nobody was already an expert, everyone had to learn, and that shared process of discovery became one of the defining experiences of the project.

By the second technical defence, these roles had remained largely stable. The second defence also reflected on the team's growth in terms of communication and coordination, noting that the challenges of integrating independently developed codebases had taught every member the importance of writing clear comments, using descriptive commit messages, and communicating before making changes that affected shared systems.

By the time of the final defence, the boundaries between individual responsibilities had become more fluid than the original task distribution suggested. Team members who had completed their primary work contributed actively to areas outside their original scope, helping to debug, adjust, and finalise systems developed by others. This collective effort during the integration phase was essential to delivering a complete and stable game, and it reflects something important about how the team had grown: by the end of the project, Quatre-Quarts was not five people working in parallel. It was five people working together.

Each member of Quatre-Quarts has a key role:

Clémence GOUTAUDIER oversees the technical side of the project. She managed the server architecture and coordinated the codebase across the team, ensuring that all technical components worked together correctly. During the final phase, she took on a central role in completing and stabilising the multiplayer system, implementing room management, enemy behaviour on the server side, client handling, level transitions, and the many details required to make the game feel coherent and complete. She also wrote the technical defence reports throughout the project.

Matteo PARNIERE-BOSQUEZ oversees the artistic and visual direction of the game. He created all character sprites and animations, designed the visual identity of the company, and was responsible for all environmental and decorative assets. He also composed and recorded all original music for each world of the game. Beyond the artistic side, he developed the player movement and mechanics system, built the custom level editor used by the entire team, and contributed actively to the final integration of gameplay systems.

Yasmine DGHOUGH supervised coordination and overall project follow-up, ensuring the team maintained good communication and a shared vision throughout development. She implemented the wandering enemy AI, designed the Air level, and added the moving platforms system which plays a central role in the Air world's gameplay. She also contributed to the user interface.

Céleste LECAMP was responsible for scheduling and the website. She ensured deadlines were respected across all three development phases and created all presentation materials for each defence. She designed the Earth level and led the development of the website from its initial structure through to its final responsive version. She also contributed to the implementation of the game's user interface, particularly the setup screen allowing players to enter their name and join or create a room.

Mary Bruny NGUEDIA NGUIAZON brought strong knowledge of video game development to the team and provided valuable guidance on gameplay systems throughout the project. She implemented the enemy AI behaviour, designed the Water level, and contributed to the user interface. She also worked closely with the team during the final integration phase, helping to ensure that independently developed systems could be brought together into a single working game.



3.2 Final task distribution and responsibilities

Across the three development phases, tasks were distributed as follows:

Clémence GOUTAUDIER was responsible for the server and all multiplayer aspects of the game, including the initial client-server architecture, the room-based session system, enemy synchronisation, player damage and respawn logic, map switching, pressure plates, gates, buttons, bridges, and hazard handling. She also designed the Fire level and wrote all three technical defence reports.

Matteo PARNIERE-BOSQUEZ was responsible for all visual assets, character sprites and animations, original music composition, the custom level editor, player movement and physics, wall jumping, the combat system, and the final integration of gameplay mechanics into the multiplayer environment.

Yasmine DGHOUGH was responsible for the wandering enemy AI, the moving platforms system, the air propeller system, the spikes and the Air level design, and contributions to the user interface.

Céleste LECAMP was responsible for the website across all three phases, the scheduling and presentation materials for each defence, the Earth level design, and contributions to the user interface including the game setup screen.

Mary Bruny NGUEDIA NGUIAZON was responsible for the enemy AI behaviour system, the Water level design, and contributions to the user interface.

At the time of the first technical defence, the task distribution was described as follows: Clémence was responsible for the server and multiplayer system. Matteo had worked on the tutorial, player movement, and collisions, and was responsible for all artistic and visual elements. Yasmine had implemented the wandering AI. Céleste was managing the website, the project timeline. Mary had also worked on the enemy AI.

At that stage, each task had been developed individually and the compiled codebase did not yet exist. Bringing all of these independently developed components together into a single working game was described as a challenging but necessary step ahead.

By the second technical defence, that step had been taken, at least partially. A first merge had been achieved, and the report reflected honestly on how much more difficult the process had been than anticipated. Different coding styles, different architectural assumptions, and different local testing environments had created significant friction during integration. The second defence report named one of its key lessons explicitly: code that is well-structured and self-contained is dramatically easier to integrate than code that is tightly coupled to a specific local context.

The second defence also updated the task distribution to reflect the work done during that phase. Clémence had completed the migration to an online server and was working on merging gameplay mechanics with the multiplayer system. Matteo was integrating the combat system into the multiplayer environment. Yasmine had been working on the combat system. Céleste was making the responsive website and was continuing to manage scheduling and presentations. Mary was contributing to the user interface.

In the final phase, as described above, the boundaries between roles became more fluid. The integration work required every team member to engage with parts of the codebase that were not originally theirs, and the daily coordination calls ensured that this cross-boundary work was managed deliberately rather than happening by accident. The final task distribution is therefore best understood not as a set of fixed individual responsibilities but as a collaborative effort in which each person's primary contribution remained clearly their own whilst the spaces between those contributions were filled collectively.

4. Level Design

4.1 – A1 Level "Earth"

The Earth world was one of the first levels to be conceptually defined and one of the last to be fully completed. Its development traces a path from a simple design idea to the most deliberately complex navigational experience in the game.

By the second technical defence, the report described its primary inspiration as the structure of ant colonies: complex underground networks of tunnels and chambers extending vertically and horizontally through the ground. This natural architecture had influenced the layout of the level, with narrow passages, branching paths, vertical drops, and rooms connecting to one another in ways that were not immediately legible. The second defence also introduced the visual identity of the world, with textures and assets evoking soil, stone, and the quiet darkness of a subterranean environment. The report noted that the level was designed to encourage players to move not just horizontally but upward and downward through the space, and that the idea was to recreate the sensation of exploring a natural underground structure.

The Earth world is probably the most deliberately disorienting level of Queen Amann. Designed not to guide the player efficiently through a space but to make them feel genuinely lost underground, and to reward patience, observation, and memory over speed and instinct.

The initial inspiration for the Earth world came from the structure of ant colonies. The complex underground networks of tunnels and chambers which extend vertically and horizontally through the ground. This natural architecture played a role in the layout of the level: narrow passages, branching paths, vertical drops, and rooms that connect to one another in ways that are not immediately legible. The visual identity of the world reinforces this feeling of being underground, with textures and drawings created by Matteo that evoke soil, stone, and the quiet darkness of a subterranean environment.

The first sub-map introduces the player to the world and functions primarily as a maze, establishing the core feeling of underground exploration. Subsequent maps develop this further, introducing rooms that either lead forward or loop back to previously visited areas. The third map shifts the design philosophy slightly towards vertical exploration, adding a sense of depth and encouraging players to think not just horizontally but upward and downward through the space.

The central mechanic of the Earth world is its door system. Multiple doors are present throughout the level, and not all of them lead forward. Some return players to earlier maps,

others lead to dead ends, and the infamous split at the fifth map presents two doors side by side, one of which leads to a small room with a handful of enemies and returns directly to the same junction, whilst the other continues towards the end of the level. Players who rush through will inevitably find themselves retracing their steps, returning to rooms they recognise with a growing sense of having been here before.

Signs are placed throughout the Earth world, but unlike in other levels they serve a dual purpose. Some signs provide genuine hints, nudging observant players in the right direction. Others are deliberately misleading, pointing towards dead ends or false paths as part of the level's design philosophy. Players cannot simply follow instructions, they must read carefully, think critically, and decide for themselves whether to trust what the environment tells them.

This balance between guidance and deception gives the Earth world a distinctive personality. Two players navigating a confusing space together are far more effective than one alone. In this way the maze structure itself becomes a cooperative mechanic.

The world concludes when players find their way to the seventh sub-map and reach the chest containing the Earth world's recipe fragment, a reward that feels genuinely earned after the winding journey through the underground.



Figure 1 - Earth level



Figure 2 - Earth level door

4.2 – A2 Level "Air"

At the time of the second technical defence, the Air level had been assigned to Yasmine and was described in broad terms. The report noted that it would be designed around the theme of air, with wind playing a central role in the player's movement and progression. The first defence described upward air currents allowing players to rise and reach higher platforms, cloud platforms some stable and some moving left to right, and hurricane-like gusts forcing players to time their jumps precisely. The combination of vertical wind mechanics, unstable cloud platforms, and timed hazards was described as intended to create a dynamic gameplay experience centred on control, timing, and adaptation.

For this level, we wanted to create a world that felt light, vertical, and dynamic. The main idea was to design a level where the player does not simply move forward on stable ground, but progresses through height, movement, and wind. We wanted the player to feel as if they were crossing the remains of an ancient place suspended in the sky, where the environment itself is unstable and shaped by air. This is why the level is built around jumps, separated platforms, vertical sections, and wind-based movement.

The inspiration for this level came from ancient air temples. We wanted to recreate the feeling of a temple built high above the ground, surrounded by wind, clouds, and open space. To support this idea, the level uses white tiles and pale blocks to represent the ruins of an old temple. These tiles give the impression of broken stone structures, abandoned platforms, and fragments of architecture that have been damaged over time. The floating islands in the background also help reinforce the feeling that the level takes place in an air realm, far from the ground.

To connect the visual theme with the gameplay, we implemented air propellers. Some of them push the player upward, allowing them to reach higher platforms and creating vertical progression. Other propellers push the player horizontally to the right, creating side wind currents. This makes the air mechanic more varied, because the player can use wind in different ways depending on the situation. In some parts, the wind helps the player rise into the sky, while in others it helps them cross gaps or move through horizontal sections.

We also added animated wind particles to make the air currents visible. This is important because the player needs to understand where the wind is and in which direction it moves. Instead of relying only on text instructions, the animation gives a clear visual clue. It also makes the level feel more alive, because the wind is constantly moving and becomes part of the environment rather than just an invisible mechanic.

The structure of the level also supports the air theme. Many areas are made of small separated tiles, which forces the player to jump carefully from one platform to another. This creates a feeling of instability and reinforces the idea that the temple is ruined and broken apart. Some sections are more vertical, while others require precise horizontal movement. Moving platforms were also added to create timing-based challenges and to make the level feel less static. The

player has to observe the rhythm of the platforms, time their jumps, and use the wind currents to progress.

Overall, the Air level was designed around the idea of a ruined sky temple where wind is both part of the atmosphere and part of the gameplay. The white tiles, floating islands, small separated platforms, vertical sections, air propellers, wind animations, and moving platforms all work together to create a level based on height, movement, and the direction of the air. The goal was to make the player feel like they are progressing through an ancient air realm by using the wind as both a guide and a challenge.



Figure 3 - Air level



Figure 4 - Air level door

4.3 – B1 Level "Fire"

At the time of the second technical defence, the Fire level had been assigned to Clémence and was described in broad terms. The report noted that the level would be centred around fire and would introduce environmental mechanics based on the interaction between dissimilar materials and the unique abilities of each playable character. The key design idea was already in place:

Tonatiuh, the inflamed skeleton, would be immune to fire and lava, whilst Citlali would not share this ability. This asymmetry would create situations requiring players to cooperate to overcome environmental challenges. The setting was described as inspired by desert landscapes associated with Latin American regions, with visual elements including sand, desert terrain, temple structures, and vegetation.

The Fire world is spread across two maps and is built around a single core design idea: the two characters do not experience the same environment. Tonatiuh, the inflamed skeleton, is immune to lava and fire. Citlali is not. This asymmetry is not merely a visual detail, it is the foundation of every cooperative challenge in this world.

The setting is inspired by a church consumed by a volcanic eruption, blending European medieval architecture with the scorched and broken aesthetic of a world overtaken by fire. The background layers feature churches and ruins receding into a warm sunset sky, giving the level a distinct and atmospheric visual identity that sets it apart from the other worlds.

Throughout the level, lava zones serve as barriers that only Tonatiuh can safely cross. This forces the two players to constantly divide responsibilities: Tonatiuh ventures into hazardous areas inaccessible to Citlali, reaches buttons and mechanisms hidden behind fire, and activates bridges that allow Citlali to advance safely. The roles then reverse, as those same bridges can isolate Tonatiuh, requiring Citlali to free them in turn. Neither character can progress without the other. Communication is not optional, it is the mechanic.

Beyond the cooperative hazards, the level also plays with player perception. Certain paths that appear safe or navigable are in fact traps, designed to mislead players and punish hasty movement. These illusions encourage the players to slow down, observe their surroundings carefully, and communicate before acting; it reinforces once again that rushed individual decisions are more dangerous than considered cooperative ones.

The level is populated with enemies across both maps, increasing pressure on the players as they navigate the fire hazards, cooperative puzzles, and deceptive terrain simultaneously. The environment also features signs that provide narrative context, grounding the volcanic devastation in the broader story of Queen Amann's curse.

The second map of the Fire world continues and deepens these mechanics, culminating in a chest containing the recipe fragment that players must note down before returning to the Lobby. The return door at the end of the second map brings players back to the hub, one step closer to completing the legendary recipe.



Figure 5 - Fire level



Figure 6 - Fire level door

4.4 – B2 Level "Water"

The Water world was the last of the four levels to be fully integrated into the game, and its design reflects the most complete expression of the project's narrative ambitions alongside its platforming mechanics. At the time of the second technical defence, the Water level had been assigned to Mary and was described as the first level of the game.

Throughout the game, each level is designed around a natural element, creating a strong contrast between environments and guiding the player's attention as they progress. The water level follows this principle by introducing an entrance door made of rocks and two amphoras on top. This design choice aligns with the artistic direction of the level, which is inspired by the architecture of an underwater Buddhist temple.

The game's core idea is that people from different cultures can connect through food. In many Asian cultures, water symbolizes purification, transition, and rebirth. Using a water-based door reinforces this symbolism, turning the entrance into a sacred passage rather than a simple doorway.

The entire water level is built around the concept of a submerged Buddhist temple. This setting naturally carries themes of spirituality, purification, and inner transformation, which directly support the player's objective: collecting a key ingredient needed to prepare the Kouign Amann that will host the queen's soul. The environment is meant to feel like a sacred journey. The ingredient found here is not treated as a simple cooking component; it is infused with the spiritual energy of the temple. The calm, meditative atmosphere of the submerged sanctuary gives the ingredient a symbolic purifying quality, making it essential for transforming an ordinary pastry into a vessel capable of containing a soul.

The architecture reinforces this idea. Nature reclaiming the temple, the flowing water, and the serene layout all evoke transcendence, suggesting that the player is gathering something more than a physical object — they are collecting spiritual essence. By choosing an underwater Buddhist temple, the level gains deeper narrative meaning and remains fully aligned with the story. The ingredient absorbs the sacred resonance of the place, becoming worthy of contributing to the queen's new host.

Mechanically, the water level stands out because its architecture is built almost entirely above the water's surface. Although the level is themed around water, the player must avoid touching it at all costs, as falling into the water results in instant death. This constraint transforms the environment into a demanding platforming challenge, requiring the player to navigate narrow paths and suspended structures with precision. The layout encourages the player to apply — and sometimes master — the movement techniques learned throughout the game. The contrast between the serene, spiritual atmosphere of the temple and the high-risk traversal mechanics creates a distinctive tension: the environment appears calm and sacred, but the gameplay demands accuracy, timing, and control. This makes the level both mechanically memorable and thematically meaningful.



Figure 7 - Water level



Figure 8 - Water level door

4.5 – The Lobby

At first, the Lobby served as a tutorial for newcomers players, where they could easily get familiar with the core gameplay mechanics, such as wall-jumping and attacking. After completing the Tutorial, players would go through a door to a level selection menu. Although the current Lobby hosts everything needed for players to understand their environment, we ultimately decided to scrap the enemies from this level, to welcome players in a peaceful land. We also decided players would access the different worlds directly from the Lobby, making their short paths to the doors a chance to get acquainted with the movement mechanics, and encouraging exploration to find new doors and objects to interact with.

The Lobby is the starting point of every session. When a player creates a room, they spawn in the Lobby. When a second player joins an existing room, however, they spawn directly in whichever map the first player is currently in, ensuring that the two players are always together regardless of where in the game they happen to be.

The Lobby serves as both a practical hub and a narrative space. It is not simply a menu dressed as a level ; it is a place worth exploring. Two signs greet players near the spawn point: one

explains the basic controls and mechanics of the game, and the other sets the scene, telling Citlali and Tonatiuh that Queen Amann has vanished, that her recipe holds the key to breaking the curse, and that four realms each guard a missing ingredient quantity. Players are invited to venture into each world, reach the end, and note down what they find.

The four doors leading to the worlds are spread across the Lobby map and require some exploration to locate. When a player stands close enough to a door, a prompt appears on screen indicating which world it leads to and inviting them to press the interact key to enter. This gentle discovery encourages players to explore the Lobby rather than rushing immediately into a level.

Visiting the lobby also rewards the player with getting acquainted with the movement possibilities with their character such as with the double-jump or the wall-jump. The players will also be able to get familiar with some game mechanics, for instance, there exists water hazards and moving platforms, which are essential to understand to overcome the four worlds guarding the recipe.

Hidden within the Lobby, and only found through exploration, is the statue of Queen Amann herself; a proud stone figure wearing a crown and holding a staff, rendered entirely by hand. Players who venture far enough will encounter her, and standing close enough triggers a prompt to interact. This opens the recipe popup, where players can enter the ingredient quantities they have collected across the four worlds. If the quantities of butter, flour, water, and sugar are all correct, a final illustration is displayed: Citlali and Tonatiuh standing alongside the restored Queen Amann. The curse is broken, the recipe is complete, and the game is won. Players may then continue to explore freely if they wish.

The Lobby thus bookends the entire experience, it is where the adventure begins, where the story is introduced, and where it ultimately concludes.



Figure 9 - Lobby



Figure 10 - Lobby door

5. Technical Implementations

5.1 Multiplayer and networking

The multiplayer system is one of the most technically complex components of Queen Amann, and its development spans the entire length of the project. Understanding its final state requires understanding the path that led to it.

At the time of the first technical defence, the multiplayer architecture had already gone through one significant redesign. The team had initially considered a peer-to-peer approach, which seemed simpler to implement and more accessible for a first attempt at networked gameplay. In practice, however, the peer-to-peer model revealed several fundamental limitations. It required one player to act implicitly as a server, which introduced questions about connection initialisation, synchronisation, and security. Testing on a home network, where ports could be configured freely, produced a working prototype, but when the code was shared with other people it failed to function on their networks. The model was also found to have security vulnerabilities that made it unsuitable as a foundation for a shared multiplayer experience.

For these reasons, the architecture was redesigned around a dedicated server before the first defence. The first defence report described this shift as a key learning step in understanding networking principles applied to video games. At that stage, the server was running locally, meaning both players had to be on the same network to connect. A chat application had been developed as an initial prototype to validate the message transmission and reception logic before adapting it for game-specific data. The first working version of the multiplayer system could transmit player positions, display names, and basic actions between two clients, but enemies were not yet synchronised and the server was not yet accessible from outside a local network.

By the second technical defence, the server had been moved online via the Koyeb hosting platform. This was a significant transition that required understanding not only how to deploy a Python application to a cloud service but also how to manage the continuous operation of a server that could not be manually restarted during a gameplay session. The second defence report described the server as receiving gameplay data from each player and redistributing it to the others, with player positions, actions, and display names synchronised in real time. However, enemy behaviour was still computed locally on each client at this stage, which meant that the two players could see different enemy reactions depending on their local game state. The report identified two potential solutions; server-controlled enemies or client-local enemies with remote detection and noted that further experimentation would determine the most efficient approach.

The multiplayer system is built on a client-server architecture implemented entirely in Python, using the Sockets library for network communication and the Threading library to handle concurrent connections. All game data exchanged between clients and the server is serialised using JSON and transmitted in binary format through a custom encoding and decoding system. Each message carries an action identifier which is processed through an event-based decorator system, allowing raw network data to be translated directly into in-game actions.

The server is deployed on the Koyeb cloud hosting platform, meaning it runs continuously without requiring any manual intervention from the team. Players can connect and play at any time without anyone monitoring or launching the server. The only situation requiring manual action would be an unexpected server crash, which the current implementation is designed to avoid. It is worth noting that the server will be kept live for a limited period following the project's final presentation.

The server runs a game loop at 60 ticks per second, handling enemy position updates, player damage calculations, and respawn logic independently of the clients. This deliberate choice ensures that game-critical logic is computed centrally and consistently, keeping gameplay smooth and preventing individual client performance from affecting the shared game state.

When a player disconnects unexpectedly, the room remains alive for any remaining connected players. The disconnected player may rejoin using the same room code, though they will respawn at the beginning of the current map. A room is only closed and removed from the

server when all players have left, ensuring that a single disconnection never destroys an ongoing session. This design is particularly important given that the game requires two players. Losing the room on a single disconnection would make the experience needlessly fragile.

All data exchange was designed with care to avoid overloading the server. Only meaningful state changes are transmitted like player positions and actions are sent only when they differ from the previously sent values, reducing unnecessary network traffic whilst maintaining real-time synchronisation.



Figure 11- Multiplayer

5.2 Room-based session system

The room system allows two players to connect to the same game session without any prior server-side configuration. When a player launches the game, they are prompted to enter a display name and either create a new room or join an existing one.

Creating a room generates a unique four-character alphanumeric code, such as "AB3X". This code is intentionally short so that players can share it easily with their partner, by message or simply by voice. The player who creates the room is assigned the first available character skin and spawns in the Lobby. The second player enters the room code to join, is assigned the

remaining skin, and spawns directly in whichever map the first player is currently in, ensuring the two players are always together from the moment the session begins.

The room system supports a maximum of two players, reflecting the cooperative two-character design of the game. Each room maintains its own independent state, including the current map, player positions, enemy states, open gates, active bridges, and plate activations. Multiple rooms can run simultaneously on the server, meaning several pairs of players can play independently at the same time without any interference between sessions.

When a room becomes empty, that is, when both players have disconnected; it is automatically deleted from the server, freeing up resources and keeping the server clean without any manual maintenance.

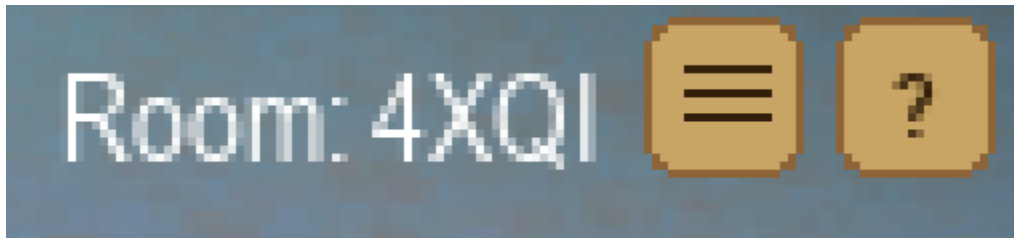


Figure 12 - Room name

5.3 Player mechanics

The player movement system forms the foundation of the entire gameplay experience. It was designed to be simple to learn whilst offering enough depth to support the increasingly complex level designs encountered across the four worlds.

Horizontal movement is performed using two directional keys, and gravity is applied constantly to the player character, pulling them downward whenever they are not standing on a surface. The jump mechanic temporarily counteracts this force, and players have access to a double jump, allowing a second jump to be performed whilst already airborne. From a platform, a player therefore has access to four jumps in total, giving them considerable freedom of movement in open platforming sections. The one restriction is that a double jump cannot be performed immediately after a wall jump, preventing unintended movement exploits whilst keeping the system intuitive.

Wall jumping allows the player to push away from any vertical surface in the game, regardless of the tile type. This mechanic adds a significant layer of mobility and is particularly useful in the more vertical sections of the Earth and Air worlds. It also opens up creative movement options for experienced players who wish to navigate levels more efficiently. It is worth noting that wall-jumping gives players a brief boost in horizontal speed, which can be exploited to jump significantly further.

The combat system allows the player to perform an attack that triggers an animation and generates an attack zone in front of the character, oriented in the direction the player is currently facing. The system checks for the closest enemy within that zone and applies damage to it. Each attack animation can damage an enemy only once per cycle, ensuring that combat feels consistent and readable. This design was driven in part by technical constraints, but it produces a clean and fair combat interaction that suits the accessible nature of the game.

Enemies deal relatively little damage compared to the damage players can inflict on them, keeping combat from feeling punishing and allowing players to focus on navigation and cooperation rather than on moment-to-moment survival. Player health does not regenerate during a level. It is only restored upon respawn, which occurs automatically when the player's health reaches zero, when they fall below the playable area of the map, or when they transition to a new map. In all cases, the player respawns at the beginning of the current map. This encourages players to be careful but never blocks progression entirely, as the game can always be continued from the map's starting point.

A short invincibility window is applied after each hit, preventing the player from losing health continuously when in close contact with an enemy. This gives players enough time to react and move away from danger without feeling overwhelmed, whilst keeping combat encounters tense enough to remain meaningful.

5.4 Cooperative mechanics

The cooperative mechanics of Queen Amann go beyond simply requiring two players to be present. They are designed so that progression is physically impossible without active communication and coordination between both players.

Pressure plates and gates

Pressure plates are triggers embedded in the environment that require both players to be standing on them simultaneously in order to open an associated gate. There are two possible configurations: either both players must stand on the same single pressure plate at the same time, or two separate plates exist in different locations and each player must stand on one of them simultaneously. The second configuration is particularly demanding, as it requires players to split up, locate their respective plates, and coordinate their actions across a distance without necessarily being able to see one another.

The game communicates the state of a pressure plate visually and intuitively. When no player is standing on a plate, it appears in a dull greyish yellow. When one player stands on it, it turns bright yellow, signalling that activation has begun but is not yet complete. When both players are correctly positioned and the condition is met, the plate turns green and the associated gate dissolves with a particle effect. This colour progression allows players to understand the mechanic through observation and experimentation rather than through explicit instruction.

The particle effect that plays when a gate opens is colour-coded to match the world the players are currently in. In the Fire world the particles are red, in the Earth world they are green, in the Air world they are a warm sandy yellow, and in the Water world they are blue. This is a deliberate artistic choice, reflecting the team's desire to push PyGame's visual capabilities as far as possible and to make each world feel visually consistent down to its smallest details.

Once a gate has been opened it remains open for the entirety of the session on that map. It will only reset if the players leave and return to the map, at which point the gate returns to its closed state. Dying, falling, or respawning within the same map does not reset a gate, meaning players are never punished for failing a section by losing cooperative progress they have already achieved.

Buttons and bridges

Buttons and bridges operate on a different and complementary logic to pressure plates. Rather than requiring sustained presence, a button is activated with a single interaction and toggles the state of its associated bridge. Two button types exist: one that opens a bridge and one that closes it. A bridge remains in its current state until the corresponding button is pressed again, meaning players do not need to remain stationary to keep a bridge active.

This system introduces a dynamic and reversible form of cooperation. A bridge can appear and disappear as many times as needed, and if one player is standing on a bridge whilst the other presses the close button, the bridge disappears beneath them and they fall. This risk adds a layer of tension and communication to the mechanic, players must announce their intentions before acting.

Buttons and bridges are used most prominently in the Fire world, where they are central to the asymmetric cooperation between Tonatiuh and Citlali. Tonatiuh can cross lava to reach a button inaccessible to Citlali, opening a bridge that allows Citlali to advance. The same bridge may then block Tonatiuh's return, requiring Citlali to find the corresponding close button to free them. This creates a continuous back-and-forth of dependency that makes the Fire world's cooperative design particularly rich and satisfying.



Figure 13 - The Bridge

5.5 Environmental mechanics

Each world of Queen Amann introduces environmental mechanics that shape how players move through and interact with the level. These systems work alongside the cooperative mechanics to create a gameplay experience that feels distinct in each world whilst remaining consistent with the core engine.

Moving platforms

Moving platforms were originally designed for the Air world, where they play a central role in the floating ruins aesthetic and the vertical exploration that defines that level. However the mechanic proved enjoyable and versatile enough that it was carried across into other worlds as well, appearing throughout the game in various contexts. Platforms move either horizontally or vertically (there are no diagonal variants) and their movement is continuous, reversing direction when they reach the end of their defined path. Players must time their jumps carefully to land on them and can ride them to reach otherwise inaccessible areas.



Figure 14 - Moving Platforms

Air current propellers

Air current propellers are environmental elements that generate a vertical wind zone above them. When a player enters this zone, they are pushed slowly but continuously upwards, rising higher and higher until they reach the maximum height of the current, at which point gravity gradually takes over and they begin to descend very slowly. The mechanic rewards patient and observant players who learn to use the currents to reach elevated platforms without expending their jumps. However, when a player jumps whilst already within an air current, the upward force combines with the jump velocity, launching them significantly higher and faster. This interaction allows skilled players to use propellers as launchpads for reaching the highest points of the Air world.



Figure 15 - Air propeller

Hazards

Three active hazard types are present in the final game: water, lava, and spikes. Water is an instant death hazard, any contact kills the player immediately and triggers a respawn at the

beginning of the current map. Spikes deal repeated small amounts of damage rather than killing instantly, giving players a brief window to escape before their health is depleted. A cooldown system prevents spike damage from being applied every frame, ensuring that brushing against a spike feels punishing but survivable rather than immediately lethal.

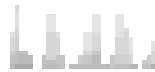


Figure 16 - Spikes

Lava immunity

Tonatiuh's immunity to lava is the only character-specific environmental mechanic in the game. Citlali has no equivalent unique ability, but as the character accessible to all players she serves as the primary perspective through which the game is experienced. Tonatiuh's immunity is not simply a passive trait -it is an active design tool. It creates situations where the two characters must approach the same environment in fundamentally different ways, making the asymmetry between them feel meaningful rather than cosmetic. This immunity is handled server-side, ensuring that it cannot be circumvented and that both players always experience the same authoritative game state.

It is worth noting that quicksand, which was planned during the design phase and mentioned in the second technical defence, was ultimately not implemented in the final game. In its place, the Fire world uses visual traps; paths and platforms that appear safe but lead players into hazards; it achieves a similar design goal of punishing hasty movement through environmental deception rather than through a physics-based mechanic.

5.6 World navigation

Navigation between maps is handled through a door system that is simple for players to understand but technically precise in its implementation. Doors are placed at the end of each map and lead either to the next sub-map within a world, to a new world entirely, or back to the Lobby. When a player stands close enough to a door, a prompt appears on their screen indicating which world or map the door leads to and inviting them to press the interact key to enter. This prompt is rendered locally on each client, meaning each player sees it independently when they approach a door themselves. Similarly, signs and text boxes are read locally, meaning that a player must approach and interact with them individually, and the other player will not see the same text unless they do the same.

When a player enters a door, the map switch is processed on the server and applied to the entire room. Since the server maintains a single shared map state for all players in a room, both players are transported to the new map simultaneously. A player cannot enter a world alone whilst their partner remains behind, the two are always kept together.

All map transitions are accompanied by an iris effect, it is a circular wipe that closes to black before the new map loads, then reopens to reveal the new environment. This effect is purely aesthetic, chosen to give transitions a cinematic and polished feel that suits the visual identity of the game. The same iris effect is used when a player dies and respawns, giving death a visual weight without interrupting the flow of gameplay for too long.

There are no locked or unavailable doors in the game. Each door has its visual identity and is always accessible in principle. The only situations in which a door may be unreachable are those created by the level design itself. For instance when a gate blocks the path to a door and must be opened by both players before they can proceed. In this way, access to progression is controlled through cooperative mechanics rather than through door states, keeping the navigation system clean and consistent.

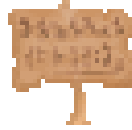
5.7 Narrative systems

Queen Amann tells its story not through cutscenes or dialogue boxes imposed on the player, but through the environment itself. Signs, chests, and the Queen statue work together to create a narrative thread that runs across the entire game, giving players a reason to explore carefully and rewarding those who pay attention.

Signs

Signs are interactive text boxes placed throughout the game world that players must approach and activate individually to read. Their purpose varies depending on the world and the intentions of the level designer. In the Lobby, signs introduce the story and explain the controls. At the entrance of each world, a sign sets the scene, describing the atmosphere and hinting at what lies ahead. Within levels, however, signs serve a richer and more nuanced role. In the Earth world for instance, signs are used not only to guide players through the underground maze but also to actively mislead them, pointing towards dead ends or false paths as part of the level's design philosophy. This narrative deception adds a layer of meaning to the text, perhaps players cannot simply follow instructions, they must think critically about what they are told.

Each sign's content was written by the member of the team who designed the corresponding level, ensuring that the tone and content of the text reflects the personality and intentions of that world's creator.

*figure 17- Sign*

Chests

A chest is placed at the conclusion of each world. Reaching it requires completing the level in its entirety, navigating all its hazards, cooperative challenges, and environmental mechanics. When a player interacts with a chest, they receive a recipe fragment not simply a number, but a narrative quantity presented in the language and atmosphere of the world they have just completed. Players must note this information down, as it will be needed later at the Queen statue. The chest therefore functions as both a gameplay reward and a narrative milestone, marking the end of each world with a sense of discovery and achievement.

*Figure 18 - Chest*

The Queen statue and the recipe finale

The Queen statue, hidden within the Lobby and only discovered through exploration, is where the game's narrative concludes. Players who have completed all four worlds and collected every recipe fragment can approach the statue and open the recipe popup, where they must enter the correct quantities of butter, flour, water, and sugar needed to bake the legendary kouign-amann.

The answers are fixed; there is only one correct recipe, and it is the same in every session. Players may attempt the recipe as many times as they wish. Each incorrect attempt is met with a gentle message reading "it doesn't taste right", letting players know they have not yet found the right combination without punishing them for trying. When the correct recipe is finally entered, a hand-drawn illustration created by Matteo is displayed on screen, showing Citlali and Tonatiuh standing alongside the restored Queen Amann. The curse is broken, the recipe is complete, and the adventure is over. Though players are free to continue exploring the game world if they wish.

This finale gives the game a clear and emotionally satisfying conclusion that ties together every level, every sign, and every chest the players encountered on their journey. It rewards curiosity, cooperation, and perseverance in equal measure.



The Secret Recipe
to make a magic Kouign Amann

Butter g

Flour g

Water L

Sugar g

... and crumbs of a Queen

Mix all of these ingredients and bake with courage.

Close Submit

Press O to close

Figure 19 - Screen to enter the ingredients



Figure 20 - Statue of the Queen Amann

5.8 Artificial Intelligence

At the time of the first technical defence, the artificial intelligence system existed as a set of independently developed components that had not yet been integrated into the main game. The first defence report described the intended long-term behaviour in detail: enemies would wander passively when the player was out of range, transition to a pursuit state when detected, and switch to an attack state when the player came close enough, before returning to passive behaviour if the player escaped. Two enemy categories had been defined (a small enemy with limited range and damage, and a large enemy with wider detection and greater impact) alongside two obstacle types, a falling obstacle and a projectile-shooting obstacle. The system was described as functional in isolation but not yet connected to the rest of the game.

By the second technical defence, the enemy system had been integrated into the multiplayer environment and was operational in the playable prototype. Enemies could detect players, pursue them, and attack. A health system had been added on both sides: players could damage enemies and enemies could damage players, with a cooldown preventing continuous damage from being applied every frame. The second defence also acknowledged the limitations that remained: enemy behaviour was computed locally on each client rather than on the server, meaning that two players could see slightly different enemy reactions depending on their local game state. Two potential solutions were discussed (server-controlled enemies or client-local enemies with remote detection) and the team noted that further experimentation would determine the most efficient approach.

In the final version of the game, the server-controlled approach was adopted. Enemy behaviour is now computed entirely on the server, which runs its game loop at 60 ticks per second and broadcasts each enemy's position, state, health, and orientation to all connected clients at every tick. Clients receive this data and render enemies accordingly, without performing any local AI computation. This ensures that both players always see the same enemy behaviour, which is

essential in a cooperative game where one player's interaction with an enemy directly affects the other's situation.

The enemy AI itself operates on the same state-based logic described in the first defence, refined and restructured into a proper class during the integration phase. Each enemy has a passive idle state, an active pursuit state, and an attack state. Transitions between states are governed by distance thresholds: when the player is beyond the detection range the enemy remains idle; when they enter the detection zone the enemy begins pursuing; when they come close enough the enemy attacks. If the player moves away, the enemy returns through the same states in reverse. Gravity is applied to enemies on the server side, and collision with the level's solid tiles is computed using the map's tile data, allowing enemies to stand correctly on platforms and terrain.

It is important to be transparent about what was not implemented. The large enemy variant, the falling obstacle, and the projectile-shooting obstacle described in the first defence were not included in the final game. The boss encounters planned during the design phase and mentioned across both previous defences (wave-based boss fights, mini-bosses guarding world chests) were also ultimately not implemented. These were genuine ambitions that the team had to set aside as the complexity of integration, multiplayer synchronisation, and level design consumed the available development time during the final phase.

What remains is a single enemy type (the chocolate bar, rendered with a full set of animations including idle, walk, attack, hurt, and death states) deployed throughout all four worlds in varying numbers and configurations. Whilst simpler than originally envisioned, this enemy fulfils its core design purpose effectively: it creates pressure, forces players to pay attention to their surroundings, and adds meaningful risk to navigation without overwhelming the cooperative focus of the game. The decision to prioritise a well-integrated, stable enemy over multiple broken or half-implemented variants was the right one, and the result is a game that feels complete and coherent rather than ambitious but unfinished.



Figure 21 - Enemy

5.9 User Interface and Menus

The user interface of Queen Amann was developed progressively across all three phases of the project, evolving from a minimal set of in-game display elements into a complete and polished system that accompanies the player from the moment they launch the game to the moment they complete it.

Setup screen and room management

The first interface a player encounters when launching the game is the setup screen. Here they are prompted to enter a display name and choose between creating a new room or joining an existing one. Creating a room generates a four-character room code that the player can share with their partner. Joining a room requires entering that code. The interface is clean and minimal, using a font consistent with the visual identity of the game, and supports both keyboard navigation and mouse interaction. All input fields and buttons are fully clickable, and the screen provides clear feedback when an error occurs, for instance when a room code is not found or when a name has not been entered.

This screen was designed and implemented by Céleste, who took Mary's original UI code as a foundation and reviewed and restructured it to ensure it integrated correctly into the game's architecture and met the standards of comfort and accessibility the team had set for the final version.

In-game interface

Once in the game, the interface provides players with the essential information they need without cluttering the screen. The player's name is displayed above their character at all times, allowing both players to identify each other easily during multiplayer sessions. A health bar in the top left corner of the screen shows the player's remaining health, changing colour from green to amber to red as health decreases, giving immediate visual feedback during combat encounters. The current room code is also displayed on screen, allowing players to share it with their partner at any time.

Help menu and settings

Two icon buttons are accessible at all times during gameplay, located in the top right corner of the screen. The first opens the help menu, which displays the current key bindings and allows players to remap them to any key they choose. This feature was implemented specifically to accommodate players using different keyboard layouts. For instance, a player on an AZERTY keyboard and a player on a QWERTY keyboard would by default have very different gameplay experiences, but with remappable controls both can play comfortably. Players can also reset all key bindings to their default values at any time.

The second button opens the settings menu, which allows players to toggle music and sound effects on and off independently. From this menu players can also return to the Lobby at any point during gameplay without needing to quit and relaunch. The visual style of both menus was designed to feel consistent with the game world, using parchment-like colours and a retro aesthetic that echoes the fonts and textures used for in-game signs, to ensure that the interface never feels disconnected from the game's artistic identity.

All menus can be navigated using both keyboard and mouse, and all windows can be closed with a mouse click rather than requiring a specific keyboard interaction. This accessibility

consideration ensures that players of all levels of familiarity with keyboard-driven interfaces can use the menus comfortably.

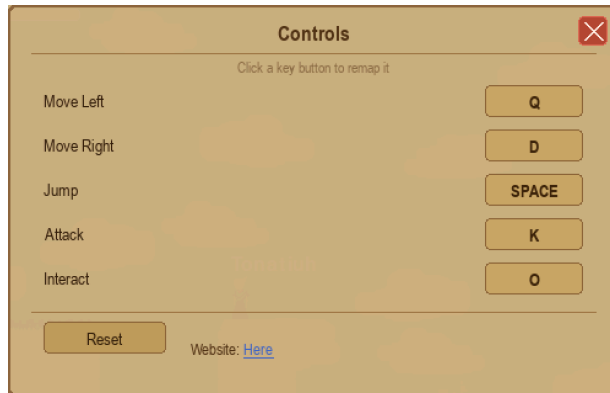


Figure 22 - Help menu

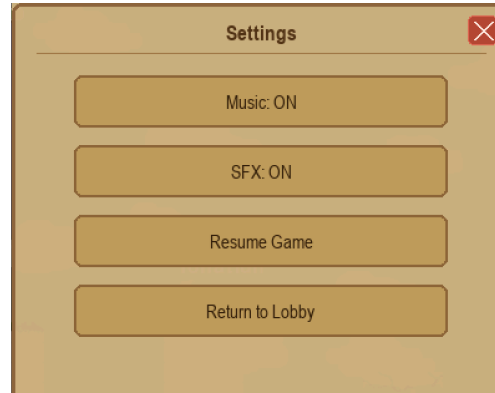


Figure 23 - Settings pannel

Level completion and narrative interface

At the end of each level, players reach a chest which triggers a narrative interface displaying the recipe fragment for that world. This interaction marks the completion of the level with a clear and satisfying reward, giving the player a tangible piece of progress to note down and carry forward into the next world. The ingredient is presented not as a bare number but in the narrative language of the world, consistent with the storytelling approach of the game.

Once all four recipe fragments have been collected, players can interact with the Queen statue in the Lobby to open the final recipe interface. Here they must enter the correct quantities of each ingredient. Each incorrect attempt is met with the message "it doesn't taste right", encouraging players to keep trying without punishing them. When the correct recipe is entered, a hand-drawn illustration by Matteo is displayed, showing Citlali and Tonatiuh alongside the restored Queen Amann. The game is won, though players are free to continue exploring.

The user interface of Queen Amann is not merely a functional layer sitting on top of the game; it is an active part of the narrative experience, guiding players through the story, rewarding their progress, and ensuring that every interaction feels intentional, accessible, and consistent with the world Quatre-Quarts has created.

6. Visuals and Audio

6.1 Visual direction and final assets

During the creative and ideation period of our project, we decided we would explore a medieval-styled fantasy world, with a certain focus on pastries.

To ensure a complete consistency in the visual aspect of our game, we have chosen to design our very own textures, ranging from terrain blocks to whole animations and titanic backgrounds. We wanted to achieve a medieval feeling. Therefore, three of the four levels were based on European and Asian medieval times.

We also made sure that our decorations and textures tell a story, one that you can interpret your way, inviting the player to decipher the story and lore of this bizarre yet interesting world.

Lobby

The Lobby had to be a place to welcome players, some that have been playing video games for a long time like new gamers. To design a welcoming area, we chose to take inspiration from *Yoshi's Island (1995)*. The area is very natural and decorated with little homes, showing the life in what could be a little village belonging to Queen Amann's vast kingdom. You can also see little homes belonging to citizens of the kingdom in the background, on cliffs and hills, showing a small part of the immenseness of the kingdom.

Water world

The Water world was heavily inspired by the Great Wall of China and by some episodes of *Les Mystérieuses Cités d'Or (2013)*. We find a huge stone wall with high mountains and statues, all surrounded by water, smoke and clouds. Other large islands float around our players who navigate through ancient stone floors, marked by the presence of water.

Earth world

Our first idea for the Earth world was an anthill. This idea led to the underground labyrinthic nature of the current aspect of the Earth world. There are many dead ends, traps and hidden secrets, and enemies are everywhere. This is also the world where "trap blocks" are introduced : They look slightly different, but if you step on them, you will fall right through them. We chose to make them a slightly different texture to be fair with the player and to make them notice there is something unusual about the terrain blocks, thus raising their awareness.

Fire world

With the Fire world, we were looking for a warm atmosphere, all while keeping a sinister feeling for the player, to show that now, the game has increased in difficulty. We thought of a church

with dim lighting, from the candles hanging from the ceiling. A church invaded by lava is quite an unusual view, and we knew how to use this fact to our advantage in our background decorations, creating quite an appealing view that the players can enjoy during the level.

Air world

The Air world was immensely inspired by *Avatar : The Last Airbender (2007)*. The entire level is filled with floating platforms made of pure white tiles, untouched by any impurities, contrasting with the tiles from the Water world. The clean platforms in the air represent what remains of the great Air temple, which once thrived with their white marble market, a material that cannot be soiled. The players jump through the ruins of this once great powerful place. Some blocks are overcome with vines to show how much time has passed. Some columns and platforms can be seen in the other worlds, they have surely crashed from up above after the collapse of the Air temple.

Characters

We chose to make the Queen and the enemies as pastries and sweet food to show they are the original and pure blooded citizens of the kingdom. Meanwhile, our characters are Citlali and Tonatiuh, a dandelion and a flaming skeleton. This choice is also a storytelling strategy for the player to guess through environmental details what happened for them to go on this quest. The chocolate soldiers are surely not in favor of reviving the Queen, therefore they must stop us. They are outsiders welcomed by the Queen, thus their strange appearances.

Game case artwork

The game case shows the whole ambience of the game in only one picture. Citlali and Tonatiuh are easily recognizable. In the foreground, we can see Tonatiuh falling and barely catching a Kouign-Amann, while Citlali is fighting a chocolate soldier behind our clumsy inflamed skeleton. Behind them is a dense forest in which they have traveled, and in the background, there are clouds symbolising the end of their journey. From the first glance, the player already knows what the goal is. On the back of the case, players are quickly introduced to the playable characters: Citlali the dandelion and Tonatiuh the skeleton, with a clear goal : "Save the Queen".

This way, the objective and the tone of the game are set before even launching it.

6.2 Sound effects and original music (composed by Matteo)

6.2.1 Sound effects

We chose to add sound effects that play when the players do an action to make the game more dynamic and make it feel more responsive. We noticed when playing videogames that having the sound off made us bored rapidly of the game, although it is a game we adore. We had the

same reaction while playing *Minecraft* (2009), where too many soothing sounds bored us if we were playing for long periods of time.

With these past experiences in mind, we chose to make dynamic sound effects using a sound effect setting on a synthesizer, and mashing various effects together at the right time with various recording methods.

Most sound effects were based on wind movement and collision with hard objects. They serve not only as audible indicators of an action being made, but also as a way of entertaining the players while they climb high structures or fight for their life against enemies.

We chose to make some sound effects louder than others based on the importance and the recurrence of each action which toggles a sound effect. Jumps and walking for example, are played at a lower volume than an attack or a hit from an enemy.

Opening a door, dying, or reading a sign toggle a very distinguishable sound, which allows the player to really feel like they are in the world of *Queen Amann*.

The sound effects make the world feel alive, mysterious, and reactive. Whether the player is climbing high structures, fighting enemies, exploring ruins, or reading signs, the sound design helps them feel that they are truly moving forward in the adventure.

6.2.2 Original music

We believed each world had such different atmospheres, and each deserved a unique theme, representing the state in which the world is. This choice helps immerse the players further in the game. Story and lore regarding the game may also be interpreted from the music in the different worlds.

Because of this, music became an essential part of immersion. It helps the player understand where they are, how they should feel, and what kind of story the environment is silently telling. Even without dialogue, a player can sense whether a place is peaceful, joyful, dangerous, mysterious, or tragic through the music.

Most of our music tracks were heavily inspired by C418's work on the *Minecraft* soundtracks, using clever loops and interesting instruments to captivate the player.

Lobby

For the Lobby, we set the atmosphere of the whole game, it is a calm and childlike game. The piano remains calm and repetitive, whereas the violas urge for a change, and add more depth to the song.

The violas also add an undertone, and show that beneath a calm and peaceful surface, there is something terribly serious at stake. This contrast is important because the Lobby is not only a cute starting area. It is also the entrance to a kingdom with a hidden past. The player may first see it as a safe and charming place, but the music suggests that there is something serious at stake. The calm piano excitedly welcomes the players. Meanwhile, the violas quietly whisper that something is wrong.

This makes the Lobby theme both comforting and slightly melancholic. It prepares the player for a game that may look innocent, but contains deeper lore and danger.

Water world

The Water world offers a joyful theme as the journey starts for our two heroes. Since this is the first world the players explore, the music has a more joyful and carefree feeling. It gives the impression of movement, discovery, and curiosity. The happy and carefree music is however met from time to time by a wave of worry. Our heroes are still blinded by the excitement of an epic journey beginning, but the music already shows the challenging undertones of this quest.

Earth world

While working on the level design of the Earth world, we had thought of an anthill, and of labyrinths. We had images of spirals and circles, never ending corridors, which gave us an excellent inspiration, driven by arpeggios which fall into low chords, giving a worrying feeling to the dirt labyrinths. The bridge during this song gives us a glimpse of hope with some high keys and ascending chords, before tumbling down once more, crushing our dreams of finding the exit of these tunnels, and bringing us back to the start of the song with a powerful arpeggio. A mesmerising yet terrifying spiral.

Fire world

The Fire world theme marks a major change in the game's atmosphere. By this point, the player must understand that the adventure has become more dangerous. The Fire world is not only visually more threatening; it also represents a change in rhythm, difficulty, and emotional intensity. The Fire world's theme drew heavy inspiration from the *Super Mario* games, especially from the section of Bowser's castle. There is a heavy and noticeable change in instruments, now with a church organ and cellos playing a terrifying backing track in tremolo. We immediately thought of Bowser's castle because we wanted to present a sinister change in pace and difficulty, as well as in ambience, to make the players understand that now, the trials would be more challenging.

Air world

The Air world theme creates another strong contrast. After the dark and heavy atmosphere of the Fire world, the Air world feels lighter, more distant, and more nostalgic. The change in instruments helps the player understand that they have entered a completely different world. The track is composed as a waltz, giving it a light and floating feeling, similar to the sensation of being high up in the air.

The change in instruments now offers a symphony of tubas in legato, backed by violas. The theme sounds like the memory of a fanfare, as if it once celebrated a great civilization that no

longer exists. This fits perfectly with the Air world, which represents the ruins of a once powerful temple.

As the journey ends in the Lobby, the composition has a whole new meaning. The violas that warned the players of a danger at the beginning of the journey now give a new sensation. Now that the heroes have everything they need to save the kingdom, the violas humbly welcome them with relief.

The journey can be told only through the soundtrack, the excitement of a new epic adventure, the doubts, the slow realization that the whole kingdom is at stake, and finally the overcoming of all the worries.

7. Website Development

The website for Queen Amann has been developed and refined across all three phases of the project, evolving from a simple presentational page into a complete, responsive, and polished platform that serves as the public face of the game and the team.

Structure and navigation

The website is organised into four main sections: Home, Documents, Gameplay and Graphics, and About Us. A fixed navigation bar is present on all pages, allowing users to move between sections without scrolling back to the top. A back-to-top feature is also implemented on longer pages, scrolling smoothly to improve the reading experience. The overall layout is consistent across all pages, ensuring a coherent and professional user experience.

From its initial desktop-only version, the website is now fully responsive, adapting cleanly to mobile phones, tablets, and desktop screens of all sizes. This improvement was completed during the second development phase and required no further correction in the final phase.

Home page

The home page was significantly updated during the final phase. The way the story of Queen Amann is told was revised and refined, giving visitors a clearer and more engaging introduction to the game's narrative and atmosphere. Importantly, the character illustrations on the home page were updated from early concept sketches and drafts to the final in-game sprites of Citlali and Tonatiuh, giving visitors an accurate and polished first impression of the game's visual identity.

Lore sections dedicated to each of the two playable characters were also added, giving visitors insight into who Citlali and Tonatiuh are, where they come from, and what motivates them. This

addition deepens the narrative presence of the website and reflects the cultural richness that underpins the game's design.

Gameplay and Graphics page

The Gameplay and Graphics page received the most significant updates during the final phase. Two gameplay videos were added: one showing an early version of the game and one showing the final version, allowing visitors to see clearly how the project evolved over the course of its development. This before-and-after presentation is an honest and effective way of communicating the scale of the progress achieved across the three development phases.

Documents page

The Documents page provides access to all project deliverables produced across the three defences, including the technical specifications document, the functional specifications document, all three technical reports, and all three presentation plans. Two download buttons were also added during the final phase, allowing visitors to download the game directly from the website for Windows, Mac OS, and Linux.

About Us page

The About Us page presents the members of Quatre-Quarts, their roles, and their contributions to the project. It also includes a contact email allowing visitors to get in touch with the team.

Overall assessment

The website fulfills all of its original objectives: it presents the game clearly and attractively, communicates the team's identity and values, provides access to all project documentation, and now also serves as a distribution platform from which the game can be downloaded directly. It stands as a complete and professional accompaniment to the game itself, and reflects the same attention to detail and creative consistency that characterises every other aspect of the Queen Amann project.

8. Project Packaging and Deliverables

USB packaging

The USB packaging was longly discussed throughout the last few months of the year. We worked closely with Enzo Bouthier to design the 3D model of the USB holder found inside the game case. After 3D printing the custom piece, we modified a DVD case and remodeled it to host the new USB holder, thus creating a perfect game case, capable of showcasing a high quality print of the game case artwork.

The USB packaging was discussed extensively throughout the last few months of the project. We wanted the final game to feel like a complete and polished product. Since we had invested a

lot of time into creating the game's universe, visuals, sound effects, music, and lore, we felt that the physical presentation of the game also had to reflect the same level of care.

For this reason, we chose to create a custom game case containing the USB key. We worked closely with Enzo Bouthier to design the 3D model of the USB holder placed inside the case. This holder had to be both aesthetic and functional. It needed to keep the USB key firmly in place, allow the player to remove it easily, and fit correctly inside the limited space of the case.

Once the model was designed, we 3D printed the custom piece. This allowed us to create a holder specifically adapted to our USB key and to the dimensions of the case. After printing it, we modified a DVD case and remodeled its interior so that it could host the new USB holder. This step transformed a standard case into a personalized game case designed for our project. The outside of the case was also important. By using a DVD case, we were able to showcase a high-quality print of the game case artwork, giving the project a more professional appearance. The cover became the first visual contact with the game and helped share its fantasy atmosphere before the player even launched it.

Inside the game case, the person holding it will find the USB stick containing the game and many important files. A physical game manual along with the installation step by step process will be provided inside the case. This way, the most essential documents will be available directly when opening the packaging.

The game case became more than a simple container : It helped turn the game into a physical object that could be presented, handled, and remembered. This made the final result feel more complete and closer to a real video game release.

Installation procedure

To ensure that Queen Amann can be installed and launched simply and reliably by anyone, regardless of their technical background, a dedicated installer was created for Windows using Inno Setup. Inno Setup was chosen for several reasons: it is free to use for non-commercial projects, it is efficient and lightweight, its interface is intuitive and straightforward to configure, and it produces a clean and professional installation experience for the end user.

The process of creating the installer involves two main steps. First, the Python project is passed through PyInstaller, which analyses the codebase and all its dependencies and compiles them into a standalone executable file. This means the end user does not need to have Python installed on their machine. Everything required to run the game is bundled into the executable. PyInstaller produces an ".iss" file associated with the application, which serves as the executable entry point.

The second step involves Inno Setup, which takes a script file with the ".iss" extension as its input. This file is a plain text script containing instructions for the Inno Setup compiler, specifying everything the installer needs to know: which files to include, where to install them on the user's

machine, what shortcuts to create, and how the installation interface should look and behave. The compiler then processes this script and produces a single standalone installer executable that the user can run to install the game in a few clicks.

The most technically challenging aspect of creating the installer was managing file paths. The path structure expected by the compiled executable differed from the original project architecture, requiring careful adjustment to ensure that all assets, maps, music files, and configuration files were correctly located relative to the executable after installation. Once these path issues were resolved, the installer was able to deploy the game cleanly and reliably.

The result is an installation experience that is fast, simple, and accessible. A user receiving the installer (whether on a USB drive or via download from the website) can have the game running on their machine within minutes, with no technical knowledge required.

9. Integration and Collaboration Challenges

9.1 GitHub workflow and lessons learned

Version control was one of the most significant practical challenges of the project, and the lessons learned from it were amongst the most valuable of the entire development experience.

The repository was organised around one main branch alongside several feature branches; one for the user interface, one for enemy testing, one for player mechanics, and one for the server. As development progressed and systems matured, the server branch gradually became the de facto main branch. This happened organically: as each feature was completed and needed to be integrated into a working multiplayer environment, it was added to the server branch and tested there. By the final phase, the server branch contained the complete and authoritative version of the game, and all active development was happening directly on it.

A significant incident occurred during the final integration phase that cost the team at least one to two days of work. A team member had been working on a separate branch for an extended period without regularly pulling the latest version of the server branch. As a result, their local version had become significantly out of date. It did not reflect the many corrections, restructurings, and new features that had been added in the meantime. When they pushed their work, it overwrote the current version with an obsolete one, removing hundreds of lines of corrections and definitions that the team had recently implemented.

What made the incident particularly difficult to resolve was that the team did not immediately identify the root cause. Several people, working simultaneously, noticed problems and began pushing their own attempted fixes, each one compounding the confusion. It was only when a previously resolved bug reappeared (one that the team knew with certainty had been fixed) that they thought to examine the GitHub commit history. By comparing the differences between commits, they were able to identify the exact push that had caused the cascade of problems. At

that point, reverting to the last good commit was straightforward, but the work the team member had contributed was lost entirely, as it had been built on top of an obsolete codebase that was no longer compatible with the current system.

The team handled the situation with maturity. Care was taken to ensure the person responsible did not feel blamed, mistakes of this kind are a natural part of collaborative development, and the incident was treated as a shared learning experience rather than an individual failure. What emerged from it was a set of clear and practical rules that governed the rest of the project.

The most important rule established was always pull before pushing. Every team member was expected to pull the latest version of the branch before committing their own work, check for merge conflicts, and resolve them before pushing. This simple discipline, consistently applied, prevented any similar incident from occurring again.

Commit messages, which had been vague and unhelpful during the first and second development phases, became a priority during the final phase. The team agreed that every commit needed a clear and descriptive message explaining what had been changed and why. This proved invaluable when debugging server-side issues: by reading the commit history on the Koyeb deployment interface, the team could correlate a broken server behaviour with a specific commit and identify the source of the problem quickly. Koyeb automatically redeployed the server with every push to the branch, and its logs including standard output and standard error streams became the primary debugging tool for server-side code that could not be tested locally.

For code that could not be tested in isolation, the team adopted a pragmatic approach: communicate before pushing. During the intense integration sessions of the final phase, team members would warn each other in real time, often during weekly Discord voice calls that frequently turned into collaborative working sessions, before pushing changes that might temporarily break the server. These calls, held at least once a week, became the backbone of the team's coordination. They allowed everyone to share their progress, redistribute tasks as needed, and work together in real time, which proved significantly more efficient than asynchronous communication alone.

The single most important piece of advice the team would offer to any group beginning a similar project is to distribute tasks frequently and in small increments rather than assigning large blocks of work infrequently. Small, focused tasks are easier to test, easier to commit cleanly, and far easier to integrate. Long periods of isolated development, however well-intentioned, create the conditions for exactly the kind of conflict that cost the team so much time.

9.2 Merging independent codebases

Beyond version control, the integration of independently developed systems into a single coherent game presented its own set of technical challenges. Each team member had worked

on their component using their own coding style, architectural assumptions, and local testing environment. Bringing these components together required not just technical skill but flexibility and a willingness to redesign work that functioned perfectly in isolation.

The most important decision made before the merging phase began was to establish structural standards for shared systems. Clémence, who coordinated the technical integration, asked Mary to rewrite the enemy system, originally developed by Yasmine and extended by Mary, into proper class definitions rather than leaving it as hard-coded logic. This refactoring, though it required additional work upfront, proved to be one of the best decisions of the project. Once the enemy behaviour was encapsulated in a clean class structure, integrating it into the server was straightforward: the file could simply be imported and the classes instantiated without requiring significant modifications to the surrounding code.

This example illustrates a broader principle that the team discovered through experience: code that is well-structured and self-contained is dramatically easier to integrate than code that is tightly coupled to a specific local context. The effort invested in generalising a system before merging it always paid off in reduced integration time and fewer unexpected bugs.

The enemy detection system provides a concrete example of the kind of adaptation that was required during integration. Originally, each client computed enemy detection locally, comparing the enemy's position to the local player's position. When enemies were moved to the server, this logic had to be redesigned: detection now needed to account for all connected players simultaneously, comparing the enemy's position to every player in the room and responding to whichever was closest or most relevant. The underlying AI behaviour (the wandering, the pursuit, the attack) did not need to be rewritten, but the context in which it operated changed fundamentally. Recognising this distinction, between what needed to change and what could be preserved, was key to completing the integration efficiently.

Throughout this phase, the team was careful to avoid duplicating effort. Tasks were assigned so that no two people were implementing the same feature in different ways simultaneously, which meant that integration conflicts were rarely about choosing between two competing implementations. Instead, the challenges were almost always about making one person's well-functioning system communicate correctly with another person's well-functioning system. It was a problem of interfaces and assumptions rather than of quality.

Looking back, the team recognises that starting with a shared architectural agreement (even a simple one) would have reduced integration friction significantly. Knowing from the outset how systems would communicate, what data formats would be used, and how the server and client would divide responsibilities would have made each individual component easier to design with integration in mind. This is perhaps the deepest lesson of the project: in collaborative software development, the hardest problems are rarely purely technical. They are organisational, communicational, and structural, and they are best addressed before a single line of code is written.

10. Objectives achieved vs Initial Roadmap

The planning of the Queen Amann project unfolded across three distinct development phases, each with its own objectives, its own challenges, and its own lessons. Looking back from the vantage point of a completed game, it is possible to trace honestly how the team's ambitions evolved, what was achieved, what was scaled back, and what the distance between initial plans and final reality reveals about the nature of collaborative development.

Phase 1 - September to January: foundations and first proof of concept

At the time of the first technical defence, the team had set itself modest but meaningful objectives: a functional server allowing two players to connect, a playable tutorial level demonstrating basic movement and collision, a working website, and a first set of visual and audio assets establishing the artistic direction of the game. Most of these objectives were met, though not without difficulty.

The server was functional in a local network configuration, but the transition to an online server had not yet been completed. The tutorial level was playable and demonstrated player movement, gravity, double jumping, and collision with the environment. The website was live and accessible. The enemy AI existed as an independently tested system but had not yet been integrated into the main game. The level editor had been built and was already in use.

What the first defence also revealed, though perhaps not explicitly enough at the time, was how much work remained. The report described a list of features still to implement that was significantly longer than the list of features already working: combat, enemy collisions, health bars, a complete gameplay loop, the room-based multiplayer system, all four world levels, environmental mechanics, and the full integration of every independently developed component into a single codebase. The team knew this, but the scale of what lay ahead was not yet fully legible.

The first defence also surfaced an organisational challenge that would persist into the second phase: tasks had been distributed and work had begun, but the connections between those tasks had not been mapped carefully enough. Each person understood their own component but had not yet had to confront the question of how it would communicate with everything else. That confrontation was coming, and it would prove harder than anticipated.

Phase 2 - January to March: integration and the multiplayer prototype

By the second technical defence, the project had advanced from a collection of separately functioning components to a genuine multiplayer prototype. The server had been moved online using the Koyeb hosting platform. Two players could connect to the same room, see each other in real time, interact with enemies, and use the combat system. Wall jumping had been added, enriching the movement system considerably. The website had become fully responsive. All four

world levels had been designed and mapped, even if not all of their mechanics were yet implemented.

The second defence report was honest about what remained. Enemy synchronisation across clients was still handled locally, meaning the two players could see different enemy behaviour. The full gameplay loop (enter a level, navigate its mechanics, reach a chest, return to the Lobby, complete the recipe) had not yet been implemented end to end. Several level-specific environmental mechanics, including air currents, lava immunity, moving platforms, and the pressure plate and gate system, were described as planned but not yet complete. The user interface beyond the basic health bar was still minimal.

The second phase also produced the project's most significant organisational lesson: the GitHub incident described in section 9.1. This event, which cost the team at least two days of work, forced a reckoning with version control practices that had until that point been treated too casually. The rules established in response to it (always pull before pushing, write clear commit messages, communicate before making changes that affect shared systems) governed the rest of the project and made the final phase significantly smoother than it might otherwise have been.

Looking back at the second defence from the perspective of the completed project, it is striking how much was achieved in the final phase that the second report described as still ahead. The gap between the second defence and the finished game is in many ways the largest of the three phases, both in terms of features implemented and in terms of the team's technical confidence and coordination.

Phase 3 - March to May: completion, integration, and delivery

The objectives set at the end of the second defence for the final phase were ambitious: complete all level-specific environmental mechanics, finalise the multiplayer system with proper room management, implement a full user interface including navigable menus, integrate all audio systems, complete the narrative finale with the Queen statue and recipe popup, package the game for delivery, and finalise all accompanying documentation.

All of these objectives were met. The final phase was the most intense of the three, but it was also the most productive, in part because the team had by then developed the habits of communication and coordination that the earlier phases had lacked. The weekly Discord calls, which became daily in the final weeks, ensured that progress was shared continuously, that blockers were identified and resolved quickly, and that no team member was working in isolation on a system that depended on another's contribution.

The one area where the final phase fell short of initial ambitions was enemy variety. Boss encounters, a large enemy variant, and obstacle types including falling blocks and projectile shooters had been described in the first defence and remained on the roadmap through the second. They were ultimately not implemented. This decision was made deliberately rather than

through oversight: the time required to design, implement, synchronise, and balance these systems on the server would have come at the cost of stability and completeness elsewhere. A single well-integrated enemy type, deployed thoughtfully across all four worlds, serves the game better than multiple partially implemented variants would have.

Similarly, the inventory system mentioned in early design discussions was not included in the final game. The team decided that requiring players to create accounts or store progress data introduced unnecessary complexity and raised data storage questions the project did not need to answer. The decision to keep the game stateless (playable from any machine, at any time, without an account) was the right one for the kind of experience Queen Amann is designed to be.

Overall assessment

The gap between what was planned at the start of the project and what was delivered at the end is not a story of failure. It is a story of a team learning, in real time, what a project of this scale actually requires. The ambitions of September 2025 were those of five people who had never built a video game before. The deliverable of May 2026 is the work of five people who have. The features that were scaled back were scaled back for good reasons, and the features that were implemented are implemented well.

What the planning history of this project ultimately demonstrates is something that no amount of prior instruction could have conveyed as clearly as the experience itself: in collaborative software development, the most important planning decisions are not made at the beginning. They are made continuously, in response to what you discover as you build. The team that finished Queen Amann plans differently than the team that started it, and that difference is perhaps the most valuable thing the project produced.

11. What We Learnt

Clémence's reflection

At the beginning of the year, I decided to take on the server and multiplayer implementation. I started by watching videos and reading through pages and pages of documentation, often without understanding what was relevant and what was not. I initially explored a peer-to-peer approach, which seemed simpler at first, and I had the chance to have friends I could call and ask questions to. I managed to get two players connected on the same local network, and I remember running to show my parents what I had built, like a child who had just done something extraordinary. But the limitations of that approach became clear quickly: when I gave

the code to other people, it did not work on their networks, because I had been testing on my own home network where I could configure the ports freely.

A friend who knew about video games helped me understand how packets were sent between computers, and I began to make real progress. After a conversation with the team, we decided to move to a proper online server, which meant starting again from a different angle. Each time I thought I understood something, there was another layer of complexity waiting. The error messages were often incomprehensible, and testing the server meant changing one line at a time and running it again, over and over, hoping something would improve.

The merging phase was one of the most technically demanding parts of the project. Matteo merged his work with Yasmine's and Mary's, and then passed everything to me to integrate into the server. Systems that worked perfectly in isolation did not always communicate the way we expected, and finding the points of incompatibility required understanding not just my own code but everyone else's as well. That kind of deep reading of other people's work is a skill I did not have at the start of this project and that I developed slowly and with some frustration over its course.

One thing I want to be honest about is the stress of the periods leading up to each defence. I found it genuinely difficult to work on my part of the project when it depended on work from others that had not yet been delivered. Sometimes things arrived at the very last minute, and because everything was interconnected, a delay in one component created pressure across the whole system. The workload did not always feel balanced, and there were moments where I felt close to being overwhelmed. I made the decision to begin writing the defence reports well in advance specifically because I knew that if I waited until the last moment, as I had done before, I would not be able to manage everything at once. That decision to take advantage, to protect my own capacity to work clearly under pressure, was one of the most useful things I did across the three phases. After each defence, once the work was visible and the effort had paid off, the feeling was extraordinary. But the path to get there was not always enjoyable, and I think that is worth saying honestly.

What I carry with me from this project, beyond the technical skills, is the understanding that in collaborative development, the hardest problems are rarely purely technical. They are about communication, timing, and trust. The more I worked on the server, the more I developed an instinct for thinking through consequences: if I do this, what happens downstream? That way of thinking did not come naturally at the start. It was built through months of making mistakes, fixing them, and trying again.

Céleste's reflection

Looking back on the project from its very beginning, the first thing that comes to mind is how quickly everything started. We had to form a group almost immediately, in a classroom where most of us were still getting to know each other. The fact that we ended up working well together

was not guaranteed; it happened because we genuinely got along, and that foundation of mutual respect and friendship carried us through the harder moments of the year.

In an ideal world, a project like this might benefit from a more deliberate distribution of existing skills across groups; pairing people who already know how to draw with people who already know how to code, so that each group starts with a broader technical base. But reflecting on it honestly, I think there is something genuinely valuable about the fact that none of us had ever built anything of this scale before. Because nobody was an expert, everyone had to learn. Nobody could simply take over a domain and leave the others behind. We all had to figure things out together, and that shared experience of learning from scratch is something that brought us closer and made the project feel truly collective.

One thing I reflect on with some care is the arrival of our fifth member partway through the ideation phase. By the time she joined, many decisions had already been made : the game concept, the characters, the general structure. She was joining a conversation that had already happened, and whilst the team welcomed her warmly, it is hard not to wonder whether she sometimes felt left out of choices she had no part in making. I hope she did not feel that way, but it is worth acknowledging honestly. A truly collaborative project ideally involves everyone from the very first decision, and that was not possible here.

In terms of timing, I think we were honest with ourselves too late about how long things would actually take. Research began in October and November but for most of the team, actual coding did not start until December. This was partly inevitable: we did not yet know enough to know what we needed to do. But it meant that in the weeks before each of the first two defences, the team was rushing to have something presentable. We always managed, and what we presented was always genuinely good, but the pressure was unnecessary and came from starting too late.

The rushing was not just stressful in itself, it created a specific kind of difficulty that I want to name clearly. Because everyone's work was interconnected, a delay in one component created pressure for everyone else. Waiting for a piece of work that had not yet arrived, knowing that your own part could not be finished without it, is a particularly frustrating experience. The workload did not always feel balanced, and there were moments where the stress was real and significant. I understand now why experienced teams set internal deadlines well before external ones, and why those deadlines need to be taken seriously by everyone, not just the people whose work happens to be most visible.

By the final phase, we had genuinely learnt our lesson. The code was largely in place ahead of time. The daily calls in the week before the final defence (at least one every day) meant that nobody was working in isolation, nobody was duplicating effort, and problems were caught and fixed quickly. That rhythm of daily communication in the final stretch was one of the best things we did as a team, and I wish we had found it earlier in the year.

There were moments of tension along the way, as there are in any group that works closely together over many months. But I think we handled them well, because we were always able to

talk. Even the GitHub incident was resolved without lasting damage to the team's dynamic. The person involved was not blamed, the situation was treated as something that had happened to all of us rather than something caused by one person, and we moved forward together.

I want to end my reflection with something that I think deserves to be said clearly. The game is incredible, and I am genuinely proud of what we built. But I want to give particular thanks to Matteo, who invested an extraordinary amount of his personal time into the music and the drawings. Every piece of original music, every hand-drawn sprite, every animated character. These are the things that made Queen Amann feel like a real game rather than a school project. His contribution transformed what could have been a technical exercise into something we are all genuinely happy to show the world.

Mary's reflection

When we first started this project, none of us really knew what direction we wanted to take. We didn't have a clear idea of the type of game we wanted to build, and since none of us had ever worked on a project like this before, everything felt a bit overwhelming. In my case, it was even more complicated because I joined the group late due to personal issues. When I arrived, the team was still trying to figure things out, and I had to catch up while also trying to understand where the project was heading.

The idea finally came from something very simple: even though we all came from different cultures, we all loved food. That was the one thing we had in common. One of us mentioned the Kouign Amann, and somehow that small comment became the spark that shaped the entire project. From there, the concept started to grow naturally. It felt like the right direction — something that represented us as a group.

Once we had the idea, we still had to decide what kind of gameplay we wanted. We spent a lot of time thinking, discussing, and sometimes going in circles. Looking back, I think the biggest lesson for me was understanding how important it is to know exactly what you want before you start building anything. At the beginning, we jumped into the project without a clear plan, and that cost us a lot of time. But eventually, we realised it and reorganised ourselves.

We divided the tasks, assigned responsibilities, and each of us started working on our part of the game. Everyone developed their section individually before merging everything together. And honestly, that merging phase was intense. Reading, understanding, debugging, and adding to someone else's code is not easy at all. It requires patience and a lot of communication. Our ACDC sessions helped a lot, because without them, I think we would have struggled even more.

After putting all the pieces together, we finally had a small prototype — not a full game yet, but something that showed the direction we were taking. From there, we had to add more components, fix bugs, and make everything work as one complete experience. There were moments of frustration, disagreements, and long debugging sessions, but we always managed to find a solution and move forward.

If I had to point out one thing we struggled with at the beginning, it would definitely be communication. We weren't always on the same page, and sometimes we worked in parallel without coordinating properly. But as the project progressed, we improved. We talked more, shared more, and slowly built a better rhythm as a team.

Reaching the end of this project wasn't easy. It took effort, patience, and a lot of teamwork. But we made it. We turned a chaotic beginning into a finished game. For me, this project was more than just coding or designing, it was about learning how to work with others, how to communicate, and how to stay focused even when things get complicated. It wasn't perfect, but we did it together, and that's what matters most.

Matteo's reflection

Looking back on the project now, I think one of the biggest lessons I would take from the whole experience is that communication is not just something that helps in a group project. It is the foundation that everything else depends on. At the beginning, we were all motivated, full of ideas, and excited to make something ambitious, but because we did not immediately establish a clear shared structure, many parts of the project became harder than they needed to be later on.

At first, we had many ideas going in every direction. This was exciting, because it showed that everyone cared about the project and wanted it to become something special. However, it also meant that we spent a lot of time discussing the general direction of the game without always making firm decisions quickly enough. In hindsight, I think we should have imposed clearer deadlines, especially through the last period of development of the game, not only for programming tasks, but also for design choices, gameplay decisions, world ideas, and technical architecture. Choosing the identity of the game is important, but it cannot take forever. To shoot for the stars, one must first climb a mountain.

The most important thing I would tell my past self is that many things seem obvious to you, but they are not necessarily obvious to everyone else. A vision that stays inside one person's head is not really a shared vision. Even when we thought we agreed on something, we sometimes had different interpretations of what that meant in practice. That created small misunderstandings at first, but later those misunderstandings became real technical and organisational problems. I think I learned that communicating your vision often, clearly, and repeatedly is essential. It avoids conflicts of ideas, prevents duplicated work, and helps everyone understand why certain decisions are being made.

One of the clearest examples of this was the architecture of the game. Looking back, the team recognises that starting with a shared architectural agreement, even a very simple one, would have reduced integration friction significantly. We should have known much earlier how the different systems would communicate, what data formats would be used, what should be

managed by the client, what should be managed by the server, and how each individual component would eventually connect to the rest of the game. At the time, it was tempting to simply start coding our own parts because that felt productive. But in reality, coding without a shared architecture sometimes created more work later.

The implementation of online features was probably the biggest moment where this became obvious. When the architecture had to change because of multiplayer and server-based logic, it shook the whole project. It forced us to reevaluate parts of the code almost from zero. Some systems that worked locally suddenly had to be reconsidered because they did not fit the new structure anymore. Moving from individual local mechanics to shared server-based mechanics changed the way we had to think about the game. We not only had to check if the game worked locally on our computers anymore, but we had to check if the mechanics worked for every player, at the same time, in the same world, without breaking synchronization.

This was difficult because some mechanics depended on other people's work, and sometimes that work was not their immediate priority. That created moments where one person could not fully progress because another system was not ready yet. In a group project, this is one of the hardest things to manage. This is simply because priorities are not aligned clearly enough. This is where better task redistribution would have helped. Decisions about redistributing tasks can and should be made earlier when we see that some work is blocked, unfinished, or not progressing fast enough. Waiting too long only creates frustration and wastes time.

Deadlines were also a major lesson. I would tell myself to impose deadlines, but more importantly, to follow up before the deadline, not only when it is already too late. Relaunching people regularly before the deadline would have avoided the rushes of work arriving all at the same time. A deadline is not useful if nobody checks progress until the final day. We needed smaller checkpoints, more frequent updates, and clearer expectations. We managed to achieve a good state during the last periods of programming. However, mastering this organisation from the start would have saved us time.

The merge incident was probably one of the most stressful moments of the project. Recovering past versions of the code after that problem forced us to focus on every line and every detail. Entire pieces of logic had been deleted, and some texture files had reverted to much older versions. It was frustrating because it felt like work that had already been completed suddenly disappeared. However, it also taught us the importance of version control discipline. We learned that saving work is not enough, we needed to protect important files, to avoid overwriting each other's progress, and to recover properly when something went wrong. We finally managed to recover our progress, but the code that was meant to be added in the first place was lost.

From a personal point of view, another difficulty was the pressure of producing many textures on very short notice while still having my own responsibilities to handle. Artistic and visual work can sometimes be underestimated because people only see the final image, not the time spent designing, testing, correcting, adapting, and making sure everything fits the game's atmosphere.

Having to design many assets quickly while also keeping up with the rest of the project was demanding.

The first defence felt very different from where we are now. At that stage, I was afraid but also excited. The project still felt open to many ambitious ideas. We had a lot of ambition, but not yet a completely stable direction. There were many ideas, and it was hard to know which ones would actually survive until the final version. I think the fear came from the uncertainty: we knew we wanted to create something good, but we did not yet fully know how we would organise ourselves to get there.

By the second defence, the feeling had changed. I was shaken by the change in architecture caused by the implementation of the online features, but I was also more confident about the advancement of the game. We could really feel that we had entered a new era for our game, with infinite new possibilities thanks to the online multiplayer implementation. Even though the technical changes were difficult, the project had become more concrete. We could see the game taking shape. We were no longer only talking about ideas; we were dealing with real systems, real problems, and real gameplay. That was stressful, but it was also proof that the game was becoming real.

Now, for the third and final defence, I feel much more confident. I am proud of the structure of the game, proud that it is now complete, and proud of the refined touches added to the presentation, mechanics, and environments. The game is not only a collection of separate ideas anymore. It feels like a finished project with an identity. The environments, mechanics, presentation, and general atmosphere have come together much more clearly. Even if the process was sometimes messy, the final result shows that the group managed to overcome many obstacles.

What I liked most about the project was seeing how separate pieces slowly became one coherent game. At the beginning, everyone had their own tasks, their own ideas, and their own way of imagining the project. But as the months passed, those separate pieces started connecting. Mechanics became playable, environments became more polished, the presentation became more professional, and the game started to feel like something we could actually show with pride. That feeling of progression was very satisfying.

I also liked the fact that the project forced us to learn things that were not purely technical. Of course, we learned about coding, online architecture, integration with said architecture, and game design. But more importantly, we learned about collaboration. We learned that a good project is not only built with good ideas. It is built with planning, communication, discipline, compromise, and structure. The hardest problems were rarely purely technical. They were organisational, communicational, and structural. They came from unclear responsibilities, delayed decisions, different interpretations, blocked dependencies, and rushed integration.

In the end, I think this project was difficult because it was ambitious. We wanted to make something complete, polished, and meaningful, not just a small prototype. That ambition created

pressure, but it also made the final result more rewarding. The problems we faced were not wasted experiences. They taught us how important preparation is, how fragile integration can be, and how essential communication is in software development.

Overall, I am proud of what we achieved. The project could have been smoother, and many things could have been done differently, but the final game represents a real evolution from our first ideas to a complete product. We started with uncertainty, many scattered ideas, and some fear. We went through architectural changes, delays, merge problems, blocked dependencies, and rushed asset production. But we ended with a game that feels structured, complete, and refined. That progression is probably the most valuable part of the experience.

The deepest lesson I take from this year is that in collaborative software development, success does not only depend on technical skill. It depends on the ability to build a shared structure before building the product itself. Before writing code, we should have agreed on communication rules, architecture, data flow, task ownership, and deadlines. If we had done that earlier, the project would likely have been less stressful and more efficient. But because we experienced those difficulties directly, we now understand their importance much better.

Yasmine's reflection

During this project, I learned a lot, both technically and personally. At the beginning, I did not fully realize how many different parts were needed to create a complete game. I first saw the project mostly as a platformer, but as we worked on it, I understood that a game is not only about moving a character. It also needs level design, collisions, animations, enemies, hazards, multiplayer synchronization, maps, assets, menus, and a coherent story.

One of the things I found difficult was working in a group. Group work means that we depend on each other, and this can sometimes be challenging. A feature can be blocked because another part is not ready yet, or a bug can appear because different parts of the project interact in unexpected ways. However, this also taught me a lot. I realized that everyone thinks differently, organizes ideas differently, and sometimes understands the same problem in a completely different way.

This made me understand how important communication is in a group project. Sometimes, we think that what we say is clear enough, but we do not always realize that it may not be clear for others. Explaining an idea, a bug, or a feature precisely is not always easy, especially when several people are working on different files at the same time. I learned that clear communication is essential to avoid misunderstandings, duplicated work, or conflicts in the code.

Another difficulty we faced during the project was using Git as a team. Sometimes, people pushed changes while still having an older version of the project, or they did not fully manage Git conflicts before pushing. This created problems because some files could overwrite newer work or bring back older bugs. It showed me that Git is very useful for collaboration, but only if

everyone understands how to use it correctly. In a group project, it is important to pull regularly, check the current version before working, communicate before pushing important changes, and resolve conflicts carefully. This was sometimes frustrating, but it also helped me understand how important version control is in real projects.

Once the basic systems of the game were implemented, the project became really enjoyable. Being able to focus more on level design made the game feel more creative and fun to build. The level editor was especially helpful, because it allowed us to place tiles, hazards, platforms, enemies, signs, and other objects more easily. Instead of changing everything directly in the code, we could build and adjust the maps visually. This made the process faster and also more satisfying, because we could immediately see how the level was taking shape.

Making a video game is fun in itself because every small feature makes the project feel more alive. Adding a new block, a new animation, a moving platform, a hazard, or a piece of scenery can completely change how the game feels. It was motivating to see the game evolve step by step from simple mechanics into something closer to a real playable world. I especially enjoyed the creative part of imagining how the player would move through a level and how the environment could guide or challenge them.

However, I also had a lot of difficulties with the enemies. The enemy sprites were not always aligned at the same height, and their position could look different depending on whether they were facing left or right. This made it difficult to make the enemy movement and attacks feel clean. I also struggled with the attack range and the visual match between the animation and the actual hitbox. Sometimes the enemy looked like it was attacking from too far away, or the animation did not perfectly match the damage timing.

Because of that, I am not completely satisfied with the enemy system. It works, but I think it could be much better with more time. I would have liked to polish the enemy animations, adjust the offsets more precisely, improve the attack hitbox, and make the behavior feel smoother and more natural. This part taught me that animation and gameplay have to match very closely in a platformer, because even a small difference in position or timing can make the game feel unfair or unfinished.

Another important thing this project taught me is how to approach a project more efficiently. At the beginning of the year, during our first meetings, we sometimes started with very broad ideas and many details at once. With time, I realized that it is better to start with a small and clear idea first, then add details progressively. For example, starting with the simple idea of making a platformer would have been easier, and then we could gradually add enemies, multiplayer, levels, story elements, hazards, and special mechanics. Trying to think about everything from the beginning can make the project harder to organize, because the vision becomes too large before the basic structure is even working.

Overall, this project helped me improve a lot. I became more comfortable reading and modifying existing code, debugging errors, using Git, and understanding how different parts of a game

interact. It also taught me that creating a game requires both creativity and precision. Even when some parts were difficult, the project was motivating because every solved problem made the game better.

12 - Conclusion

Queen Amann began as an idea without a shape, held by five people who had never built a video game before. It ends as a complete, fully playable cooperative experience with four distinct worlds, an original soundtrack, handcrafted visual assets, a live online multiplayer server, a responsive website, and a narrative that concludes with a hand-drawn illustration of two characters standing alongside the queen they set out to save. The distance between those two points is the story of this project.

Looking back across the three development phases, what is most striking is not the technical complexity of what was built, though that complexity is real and should not be understated, but the trajectory of the team that built it. The group that presented its first technical defence in January 2026 was five people working largely in parallel, each developing their component in isolation and hoping that the pieces would fit together when the time came. The group that presents this final defence is something different: a team that has learnt, through experience and sometimes through difficulty, what collaborative software development actually requires. The technical skills are visible in the game. The organisational and human skills are visible in the fact that the game exists at all.

The project was not without its struggles. There were periods of rushing before each defence, when work arrived later than expected and the interconnected nature of the codebase meant that a delay in one component created pressure for everyone else. There were moments of tension within the team, navigated through communication and mutual respect. There was a significant version control incident that cost days of work and required the team to examine honestly how they were collaborating. There were ambitions, including boss encounters, an inventory system, and a game trailer, that had to be set aside when the reality of what remained to be done made their implementation impossible without compromising the quality of everything else.

These are not failures. They are the normal texture of a first large-scale collaborative project, and the team's ability to acknowledge them honestly, learn from them, and continue moving forward is itself one of the most significant achievements of the year.

What was achieved is extraordinary for a first project. The multiplayer system, designed and implemented from scratch by someone who began the year without knowing what a socket was, runs live on a cloud server and supports simultaneous independent game sessions. The visual assets, every sprite, every background, every animated character, were drawn entirely by hand. The music, one original track per world plus a lobby theme, was composed, recorded, and integrated by a single team member working in his own time. The level editor, built specifically

for this project, was used to design every map in the game. The cooperative mechanics, pressure plates, gates, buttons, bridges, lava immunity, and the recipe finale, were designed with a clear and consistent philosophy and implemented to a standard that makes them genuinely enjoyable to play.

Playtesting confirmed what the team had hoped: the game is fun. People who play video games regularly found it engaging and well-crafted. People who never play video games found it accessible, calm, and worth exploring. A parent got lost in the Earth world maze and found it interesting rather than frustrating. Non-gamers who tried it said they enjoyed it, which is perhaps the most honest measure of whether a cooperative puzzle-platformer has achieved what it set out to do.

There are individual contributions that deserve to be named. Matteo's investment in the music and visual assets transformed what could have been a functional but aesthetically unremarkable student project into something with a genuine identity. Clémence's persistence through the complexity of the server architecture, restarting from a different approach more than once without losing sight of the goal, is the reason the game can be played online at all. Yasmine's moving platform mechanic, originally built for the Air world, became one of the most-used systems in the entire game. Céleste's management of the website, the scheduling, and the presentation materials across all three phases ensured that the project was always presented with care and clarity. Mary's contribution to the enemy system and the Water level, alongside her work during the integration phase, helped ensure that independently developed systems could become a single coherent game.

Queen Amann is our first project. It is imperfect in the ways that all first projects are imperfect, and it is extraordinary in the ways that only a project built with genuine care and commitment can be. We are proud of it. We are proud of each other. And we are grateful for everything this year has taught us, not only about Python, PyGame, networking, and version control, but about what it means to build something together.

The Queen has been saved. The recipe is complete